

Data Structures: Assignment-6

Name: Suraj Shah

Admission Number: I24AI012

Question 1: Write a C/C++ code to evaluate the Postfix Expression
using Stack.

Code:

```
#include <stdio.h>

#define MAXSIZE 100
struct stack {
    int stArr[MAXSIZE];
    int top;
};
typedef struct stack STACK;

void create(STACK *s) {
    s->top=-1;
}

int isFull (STACK *s) {
    if (s->top==MAXSIZE-1) {
        return 1;
    }
    return 0;
}

int isEmpty (STACK *s) {
    if (s->top== -1) {
        return 1;
    }
    return 0;
}

void push (STACK *s, int data) {
    if (isFull(s)) {
        printf("Stack Overflow");
        return;
    }
    s->top++;
    s->stArr[s->top]=data;
}
```

```

int pop (STACK *s) {
    if (isEmpty(s)) {
        printf("Stack Underflow");
        return -1;
    }
    int x=s->stArr[s->top];
    s->top--;
    return x;
}

int evaluate(char expression[]) {
    STACK s;
    create(&s);
    int op1,op2,res;
    for (int i=0; expression[i]!='\0'; i++) {
        if (expression[i]>='0' && expression[i]<='9') {
            int num=0;
            while (expression[i]>='0' && expression[i]<'9') {
                num=num*10+(int)expression[i++]-48;
            }
            push(&s,num);
        }
        else if (expression[i]!=' ') {
            op2=pop(&s);
            op1=pop(&s);
            switch(expression[i]) {
                case '*':
                    res=op1*op2;
                    break;
                case '/':
                    res=op1/op2;
                    break;
                case '-':
                    res=op1-op2;
                    break;
                case '+':
                    res=op1+op2;
                    break;
            }
            push(&s,res);
        }
    }
    return pop(&s);
}

int main() {

```

```
char expression[100];  
printf("Enter an expression: ");  
gets(expression);  
printf("%d",evaluate(expression));  
}
```

Output:

```
Enter an expression: 3 4 * 2 5 * +  
22
```

Question 2: Implement a queue using a linked list with two pointers:

front and rear. Elements can only be inserted via the rear pointer and deleted via the front pointer. The queue has the following basic operation:

- enqueue() – Insert an element in the queue.
- dequeue() – Remove the element from the queue.
- peek() – Return the element at the front node of the queue
- isFull() – Validates if the queue is full or not.
- isEmpty() – Checks if the queue is empty.

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
}Node;

void enqueue(Node **f, Node **r, int data) {
    Node *newNode=(Node*)malloc(sizeof(Node));
    newNode->data=data;
    newNode->next=NULL;
    if (*r==NULL) {
        *r=newNode;
        *f=newNode;
    }
    else {
        (*r)->next=newNode;
        *r=newNode;
    }
}

int isEmpty(Node *f) {
    if (f==NULL) {
        return 1;
    }
    return 0;
}

int dequeue(Node **f, Node **r) {
    if (isEmpty(*f)) {
```

```

        printf("Queue is Empty\n");
        return -1;
    }
    else {
        *f=(*f)->next;
        return (*r)->data;
    }
}

int peek(Node **f, Node **r) {
    if (isEmpty(*f)) {
        printf("Queue is Empty\n");
        return -1;
    }
    return (*f)->data;
}

int main() {
    Node *front=NULL;
    Node *rear=NULL;
    int choice, data;
    while (1) {
        printf("\nQueue Operations:\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                enqueue(&front, &rear, data);
                break;
            case 2:
                printf("Element %d deleted\n", dequeue(&front, &rear));
                break;
            case 3:
                printf("The front element of the queue is %d", peek(&front,
&rear));
                break;
            case 0:
                return 0;

            default:
                printf("Invalid choice!\n");
        }
    }
}

```

```
}  
}
```

Output:

```
Queue Operations:
```

- 1. Enqueue
- 2. Dequeue
- 3. Peek
- 0. Exit

```
Enter your choice: 1
```

```
Enter data: 10
```

```
Queue Operations:
```

- 1. Enqueue
- 2. Dequeue
- 3. Peek
- 0. Exit

```
Enter your choice: 1
```

```
Enter data: 20
```

```
Queue Operations:
```

- 1. Enqueue
- 2. Dequeue
- 3. Peek
- 0. Exit

```
Enter your choice: 1
```

```
Enter data: 30
```

```
Queue Operations:
```

- 1. Enqueue
- 2. Dequeue
- 3. Peek
- 0. Exit

```
Enter your choice: 2
```

```
Element 30 deleted
```

```
Queue Operations:
```

- 1. Enqueue
- 2. Dequeue
- 3. Peek
- 0. Exit

```
Enter your choice: 3
```

```
The front element of the queue is 20
```